Contents

Implementation Plan: Generic Raw-Data → Mesh Projection Feature

Date: 2026-05-09 **Status:** Draft v1 **Scope:** Lay down a reusable pipeline that takes a raw scalar/vector field sampled in some external coordinate system (geographic, regional UTM, arbitrary structured grid) and projects it onto the current MFEM `Mesh` / `ParMesh` so it can be consumed as **initial conditions or material properties** by the dynamic-rupture (`WaveOperator` + `FaultFaceFlux`) and quasi-dynamic (`SEASQuasiDynamicOperator`) drivers.

First concrete user: SCEC CVM-H velocity model (`safs/project_7.0_alternative/data_velocity/velocity_raw_`
— 29 ASCII slices, 143 lon × 71 lat × 29 z, fields `Vp / Vs / ρ`) onto the `safs_fault_box_nwcut_{500,1000,2000}m.msh` doubled-domain meshes.

Second concrete user (immediate follow-up): background stress tensor σ_0(x) and pore pressure p(x) on the same meshes, with the same pipeline.

Predecessor / context references:

- `document/system_dev/dynamic_rupture_plan_v4.md` (defines `WaveOperator`, `DOFData`, `FaultFaceFlux::EvaluateTotal`)
- `document/fullelasticity_dev/fullelasticity_implementation_plan_02282026.md` (defines elasticity assembly + initial state)
- `safs/project_7.0_alternative/code_preprocess/nw_cut_strip.py` (reference for the offline-preprocess pattern in this project)
- `dynamic/tpv102_setup_total.hpp::InitializeStateTotal` (current "set bulk Q" hook this feature must extend, not replace)

Overview

Today every miniapp driver hard-codes a homogeneous (λ, μ, ρ) triple at the `WaveOperator` constructor and a closed-form (σ_n, τ_ini) field at `InitializeStateTotal`. To run real-data benchmarks (CVM-H velocity, regional stress maps from CFM, lab pore-pressure profiles) we need a **generic data projection feature** that:

1. Reads a raw external dataset in its native coordinate system.
2. Converts coordinates and units to the project canon: **UTM 11 N (EPSG:32611), z = elevation in metres (z = 0 at the free surface, z < 0 at depth)**.
3. Builds a 3-D structured-grid interpolant.
4. Evaluates that interpolant at every required mesh entity (bulk DOFs, fault QPs, optionally face QPs) and writes the result as an MFEM `(Par)GridFunction` (per-field, per-mesh, per-resolution) plus a ParaView snapshot for visual QA.
5. Hands the resulting `GridFunction` to a thin **adapter** that injects the per-DOF values into the wave operator (λ(x), μ(x), ρ(x)), the fault flux (per-DOF impedances Z_p, Z_s and pre-stress σ_n0, τ1_0, τ2_0), and the friction law (per-DOF parameters that vary in space, e.g. a(x), Dc(x)).

The **same pipeline** must serve `Vp / Vs / ρ`, σ_0_components, pore pressure, and any future scalar/tensor field; the only thing that changes per-field is the schema (named columns) and the consumer adapter.

Split: **offline Python preprocessor** handles platform-specific bits (geographic CRS conversion via `pyproj`, ASCII parsing) and writes a **normalized HDF5 sidecar** keyed to a specific mesh; **runtime C++ loader** reads the sidecar, builds the interpolant, projects onto mesh DOFs, and produces `GridFunctions`.

Constraints

Interface constraints (cannot change)

- **`WaveOperator(MeshType&, int order, real_t λ, real_t μ, real_t ρ, const BoundaryConfig&)`** — the existing scalar-material constructor. Extension is by overload, never by mutation. Existing TPV102 / TPV104 / TPV205 / BP5 drivers must compile and produce byte-identical results.
- **`DOFData` layout in `dynamic/fault_face_flux.hpp`** — must remain binary-compatible. Per-DOF impedances (`Zp_plus`, `Zp_minus`, `Zs_plus`, `Zs_minus`, `eta_p`, `eta_s`) and pre-stress (`sigma_n0`, `tau1_0`, `tau2_0`) already exist as fields; the projection adapter only writes into them.
- **`InitializeStateTotal(Vector& Q, int ndof_total, real_t σn, real_t τini)`** — keep the constant-field overload as-is for TPV102; add a new **field-valued overload** that takes a `GridFunction` per σ-component.

Convention constraints

- **Canonical CRS:** UTM 11 N, EPSG:32611. All meshes in this project are in UTM 11 N (`safs/project_7.0_alternativ` carry `GOCAD_ORIGINAL_COORDINATE_SYSTEM` block; `AXIS_UNIT m m m`; `ZPOSITIVE Elevation`).
- **Z convention:** z = 0 at free surface, z < 0 at depth. Raw datasets that use "depth, positive down, in km" must be flipped at the preprocessing stage: `z_canon = −(depth_m)`.
- **File naming:** offline outputs land in `safs/<project>/data_projected/<field-set>_<mesh-tag>.h5` where `<mesh-tag>` matches the mesh stem (`safs_fault_box_nwcut_500m`). Mesh-coupled outputs are mesh-specific by construction — never reuse one project's `.h5` against a different mesh.
- **Document convention:** "no-touch BP5 source" (CLAUDE.md C2). All new consumer hooks in `dynamic/` must be additive; no edits to `bp5/`, `bp1/`, `bp2/`, `domain/`, `friction/dieterich_ruina.hpp`.

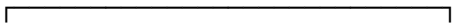
Numerical constraints

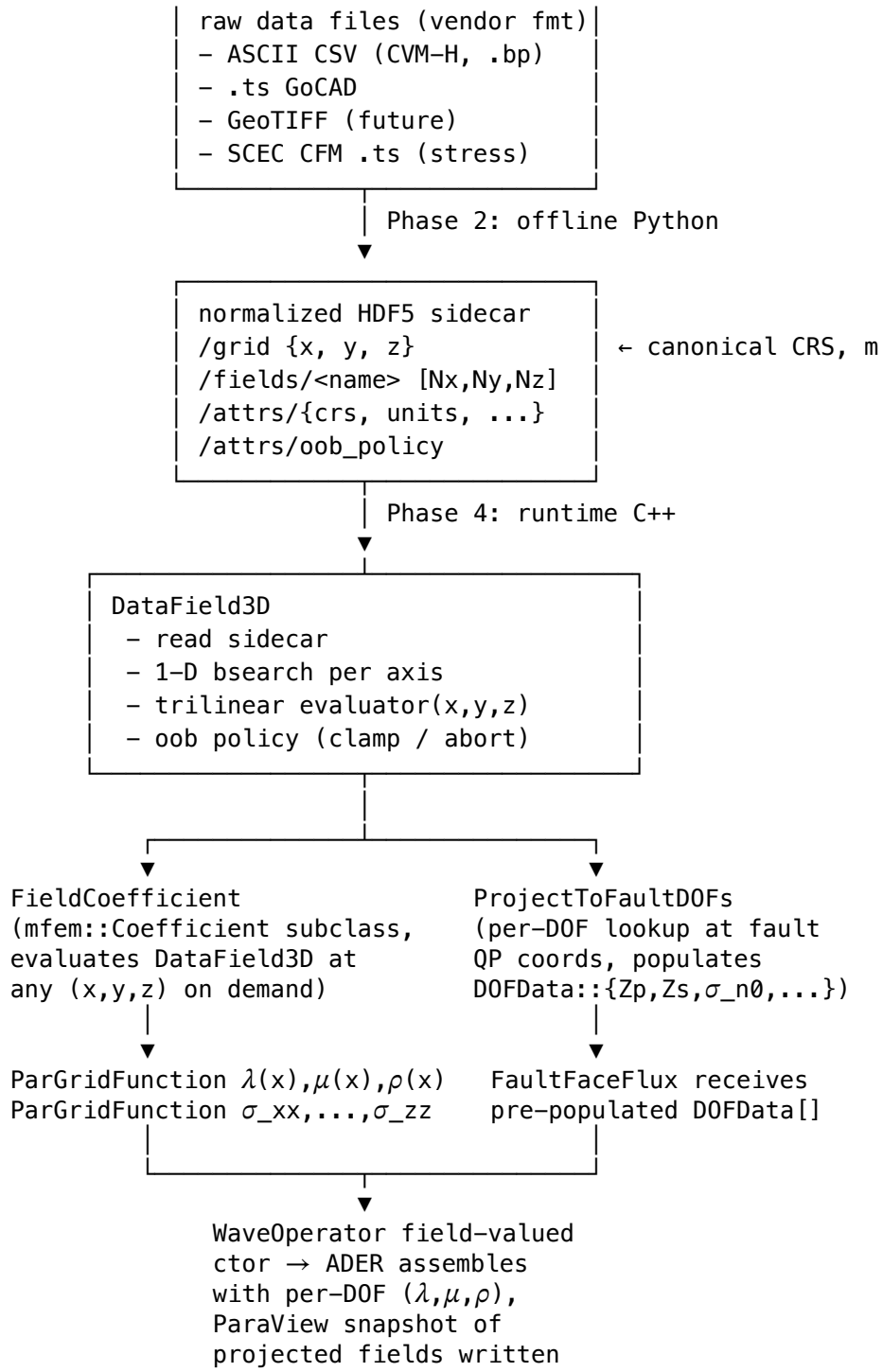
- The CVM-H velocity raster horizontal extent is **smaller** than the doubled-domain mesh footprint: lon ∈ [−118.17, −115.68], lat ∈ [33.34, 34.57] (≈ 250 × 135 km), versus mesh xy = 715 × 493 km. Out-of-coverage policy must be explicit (clamp-to-nearest by default for material fields; abort-on-miss for stress fields where extrapolation is physically meaningless).
- Vertical sampling is **non-uniform**: the CVM-H slice depths are {0, 1, 2, …, 18, 20, 22, 24, 26, 28, 30, 40, 50, 60, 70} km. The filename label `velocity_raw_X.Ykm.bp` is **misleading** (it equals `depth_in_km / 10`); the `# Depth(m): N` header is the source of truth. The interpolant must accept arbitrary monotone z-knots, not assume uniform Δz.
- Material projection must preserve $\lambda \geq 0$, $\mu > 0$, $\rho > 0$ after projection. The CVM-H `Vs Min_v: 120.69 m/s` (water-saturated near-surface cells) makes $\mu = \rho\ V\_s^2 \approx 24$ MPa, three orders of magnitude below competent rock; this is real but kills `Δt_CFL`. The adapter must expose a **floor** (default `V_s_floor = 1500 m/s`, configurable) applied at projection time and report how many DOFs were clamped.

Performance constraints

- Projection runs **once at driver startup** per mesh per field set. A single-pass O(N_DOF · log N_grid) trilinear evaluation with a 1-D binary search per axis is acceptable. No need for KD-tree / RTree.
- For 500 m mesh: ~412 k bulk tets × 4 nodes/tet × 3 fields × 8 bytes ≈ 40 MB peak memory for the projected fields — trivial.
- **MPI:** in `ParMesh` mode, every rank loads the full structured grid (≈ 10 k × 29 × 3 × 8 B ≈ 7 MB). Only the rank-local mesh DOFs are evaluated. No collective beyond what `ParGridFunction::ProjectCoefficient` already does.

Architecture





Phase 1 — Schema & Coordinate Convention

Goal

Pin down (a) the canonical coordinate system every projected field lives in, (b) the file format of the normalized sidecar, (c) the field-set schema for the three immediate users (velocity, background stress, pore pressure). After this phase nothing executes — the contract for Phases 2–4 is fixed.

Files to Create

- `miniapps/seas/document/features_dev/data_projection_schema_v1.md` — the **schema reference** (HDF5 layout, attribute names, units, dtype, CRS identifiers). Treat as part of the codebase: any breaking change bumps the schema version (v1, v2, …) and the runtime loader rejects mismatched versions.

Files to Modify

- `miniapps/seas/document/features_dev/data_projection_feature_plan_v1.md` (this file) — keep aligned with the schema doc.

Detailed Requirements

1. **Canonical CRS identifier.** The HDF5 root attribute `crs` is a string with two recognised values: `"EPSG:32611"` (UTM 11 N, the only value used by current SAFS work) and `"local-tangent"` (a reserved second value for future small-domain benchmarks; not implemented in this plan, just reserved).
2. **Canonical units.** Attribute `units = "m"` for x, y, z. Field-specific unit attributes per dataset (e.g. `/fields/Vp` carries `units = "m/s"`, `/fields/sigma_xx` carries `units = "Pa"`).
3. **Canonical z direction.** Attribute `z_positive = "elevation"` (matches the GOCAD .ts convention used by the source faults). The offline preprocessor is responsible for converting any "depth-positive-down" raw inputs to elevation before writing.
4. **Sidecar structure (HDF5).**

```
<sidecar>.h5
├── attrs:
│     schema_version = "data_projection_v1"
│     crs            = "EPSG:32611"
│     units          = "m"
│     z_positive     = "elevation"
│     created_at     = ISO-8601 timestamp
│     source         = "<comma-separated source filenames>"
│     source_crs     = "EPSG:4326"          # geographic for CVM-H
│     mesh_tag       = "safs_fault_box_nwcut_500m"  # informational
│     oob_policy     = "clamp" | "abort"
├── /grid
│     ├── x  [Nx]  float64, strictly monotone increasing
│     ├── y  [Ny]  float64
│     └── z  [Nz]  float64
└── /fields
      ├── Vp        [Nx, Ny, Nz]  float64, attrs: units="m/s"
      ├── Vs        [Nx, Ny, Nz]  float64, attrs: units="m/s"
      ├── density   [Nx, Ny, Nz]  float64, attrs: units="kg/m^3"
      └── ...                                  # any number of named fields
```

   The grid must be **structured rectilinear** (separable axes), not curvilinear. Out of scope: unstructured 3-D point clouds; if the input is unstructured, the offline preprocessor must resample onto a rectilinear grid first (Phase 2 §6).
5. **Index order convention.** All 3-D arrays use `[Nx, Ny, Nz]` row-major (C order, the HDF5 default). The runtime loader (`DataField3D::Load`) checks `axes-monotone` and `dataset.shape == (len(x), len(y), len(z))` and aborts with a precise error message if either fails.

6. **NaN / fill convention.** Cells outside the source raster (e.g. CVM-H has a non-rectangular geographic mask after re-projection to UTM) are written as NaN. The interpolant treats a stencil that contains any NaN as out-of-bounds and applies the OOB policy.
7. **Field-set schema for the three immediate users.**

Field set	Field names	Units
`velocity`	`Vp`, `Vs`, `density`	m/s, m/s, kg/m^3
`stress`	`sigma_xx`, `sigma_yy`, `sigma_zz`, `sigma_xy`, `sigma_yz`, `sigma_xz`	Pa each
`pore_pressure`	`p`	Pa

Adding a future field set does **not** require schema changes — just adds entries under `/fields`.

Acceptance Criteria

- ☐ Schema doc enumerates all attributes, dtypes, dimension order, OOB policies in one place.
- ☐ An example sidecar can be opened with `h5dump -A` and every required attribute is present.
- ☐ The schema is referenced by Phase 2 (writer) and Phase 3 (reader) requirements; both sides cite the doc.

Dependencies

- Depends on: nothing.
- Required by: Phases 2, 3, 4, 5.

Phase 2 — Offline Preprocessor (Python)

Goal

Convert the CVM-H ASCII slices in `safs/project_7.0_alternative/data_velocity/velocity_raw_*.bp` to a single schema-conforming HDF5 sidecar in `safs/project_7.0_alternative/data_projected/velocity_safs_d` After this phase the runtime loader has a deterministic input and the Phase 4 / 5 work can proceed against the sidecar without depending on `pyproj`.

Files to Create

- `safs/project_7.0_alternative/code_preprocess/data_projection/__init__.py`
- `safs/project_7.0_alternative/code_preprocess/data_projection/raw_readers.py` — one reader per raw format. Initial readers:
  - `read_cvmh_ascii(paths: list[Path]) -> RawSliceStack` — parses the CVM-H `velocity_raw_*.bp` slices.
  - `read_csv_grid(path: Path, columns: list[str], crs: str) -> RawGrid` — generic CSV reader for future stress maps (Lon, Lat, sigma_xx, …).
- `safs/project_7.0_alternative/code_preprocess/data_projection/crs.py` — thin `pyproj.Transformer`-wrapped helpers `geographic_to_utm11n(lon, lat) -> (x, y)` and `flip_depth_to_elevation`.
- `safs/project_7.0_alternative/code_preprocess/data_projection/sidecar.py` — `write_sidecar(out_path, grid: dict, fields: dict, attrs: dict)` obeying the Phase 1 schema.
- `safs/project_7.0_alternative/code_preprocess/data_projection/build_velocity_cvmh.py` — top-level driver: reads `data_velocity/velocity_raw_*.bp`, calls `geographic_to_utm11n`, packs into a 3-D rectilinear grid in UTM 11 N, writes `data_projected/velocity_safs.h5`. CLI: `--res 500`

`1000 2000` to emit one sidecar per mesh resolution if any per-mesh subsetting is desired (default: emit one sidecar that covers the largest mesh, all resolutions reuse it).
- `safs/project_7.0_alternative/code_preprocess/data_projection/test_data_projection.py` — pytest suite mirroring the `test_nw_cut_strip.py` pattern (synthetic fixtures + one real-data smoke).

Files to Modify

- None — all new code under `code_preprocess/data_projection/`.

Detailed Requirements

1. **CVM-H reader (`raw_readers.read_cvmh_ascii`).** Signature:

```python
def read_cvmh_ascii(paths: list[Path]) -> RawSliceStack:
    """Return a RawSliceStack with attributes:
      lon_grid : (N_lon,) float64, strictly increasing
      lat_grid : (N_lat,) float64, strictly increasing
      depths_m : (N_z,)   float64, monotone increasing (positive=down)
      fields   : dict[name -> (N_lon, N_lat, N_z) float64, NaN where missing]
    Each path is one slice; depth comes from the `# Depth(m): N` header
    line, NOT from the filename (filenames use a misleading `X.Ykm`
    label that equals depth_km/10 — see schema doc §3.5).
    """
```

   Parser steps per file:

   - Read the # header until the column line `# Lon,Lat,Vp(m/s),Vs(m/s),Density(kg/m^3)`.
   - Capture `Depth(m)`, `Spacing(degree)`, `Lon_pts`, `Lat_pts`, `Total_pts`, `Lat1, Lon1, Lat2, Lon2`.
   - `assert Lat_pts * Lon_pts == Total_pts`.
   - Read the CSV body with `numpy.loadtxt`. Reject malformed rows.
   - **Pin grid orientation.** The headers state `Lat1=33.3428, Lon1=-118.1677` (SW corner) and `Lat2=34.5674, Lon2=-115.6932` (NE corner). Verify the first sample row equals `(Lon1, Lat1, …)` and the last row equals `(Lon2, Lat2, …)`; abort otherwise.
   - Reshape to `(Lon_pts, Lat_pts)` with the row-major order observed in the file (lon outer, lat inner per the existing files; verify by checking that the first `Lat_pts` rows share the same lon).

   Across slices: assert all slices share an identical `(lon_grid, lat_grid)`. Stack along the depth axis sorted by `Depth(m)` ascending.

2. **CRS conversion (`crs.geographic_to_utm11n`).** Signature:

```python
def geographic_to_utm11n(lon: np.ndarray, lat: np.ndarray
                         ) -> tuple[np.ndarray, np.ndarray]:
    """Vectorised lon/lat → UTM 11 N. Uses pyproj.Transformer with
    always_xy=True and never_failed=True; raises if pyproj missing."""
```

   Critical trap: `geographic_to_utm11n` returns x = f(lon, lat) and y = g(lon, lat) — both depend on **both** inputs because UTM is not a pure cylindrical projection. So a regular geographic grid is **not** regular in UTM. The build script (§5) MUST resample onto a regular UTM rectilinear grid; it must NOT just re-label the lon/lat samples as their UTM equivalents.

3. **Resampling strategy (`build_velocity_cvmh.py`).** Steps:

   1. Read slices via `read_cvmh_ascii`.
   2. Convert every (lon, lat) pair to (x_utm, y_utm). Now we have an irregular 2-D point cloud per slice.
   3. Build a rectilinear UTM grid covering the convex hull, with $\Delta x = \Delta y =$ `grid_spacing_m` (default `1500.0` = LC_NEAR).

4. For each slice, scipy.interpolate.LinearNDInterpolator from `(x_utm, y_utm)` to the rectilinear grid; fill outside the convex hull with NaN.
5. Convert depth → elevation: `z_canon[k] = -depths_m[k]`. Sort `z_canon` ascending so `z_canon[0]` is the deepest.
6. Stack `(N_x, N_y, N_z)` per field, write via `sidecar.write_sidecar`.

4. **Sidecar writer (`sidecar.write_sidecar`).** Signature:

```python
def write_sidecar(out_path: Path,
                  x: np.ndarray, y: np.ndarray, z: np.ndarray,
                  fields: dict[str, np.ndarray],
                  attrs: dict[str, Any]) -> None:
    """Write the schema-v1 HDF5. Asserts:
      - x, y, z strictly monotone increasing
      - every fields[name].shape == (len(x), len(y), len(z))
      - attrs contains required keys per Phase 1 §4
    """
```

5. **CLI driver (`build_velocity_cvmh.py`).** Signature:

```
python -m data_projection.build_velocity_cvmh \
    --raw-dir   data_velocity \
    --out-path  data_projected/velocity_safs.h5 \
    --grid-dx   1500 \
    --vs-floor  1500   # m/s; sets attr but does NOT modify field values
    --verbose
```

   The `--vs-floor` flag only writes an attribute; the actual flooring happens at projection time on the runtime side so the sidecar stores the unaltered model.

6. **Tests (`test_data_projection.py`).** At minimum:

   - `test_read_cvmh_ascii_orientation` — fabricates a 3 × 2 × 2 fixture with known `Lat1/Lon1/Lat2/Lon2` and asserts the parsed `lon_grid` and field array map back to those corners.
   - `test_filename_depth_label_is_ignored` — fixture file named `_5km.bp` with `# Depth(m): 50000` returns z = 50 000 (mismatches are common in the real data; this guards against regression).
   - `test_geographic_to_utm11n_round_trip` — points round-trip within 0.1 m.
   - `test_resample_preserves_constant_field` — feed Vp ≡ 4000 over the whole hull; assert all in-hull UTM grid points have Vp = 4000 within 1e-9, all out-of-hull points are NaN.
   - `test_sidecar_round_trip` — write → read with h5py and check every attribute / dataset matches.
   - `test_smoke_real_data` (skipif `data_velocity/` absent) — runs the full pipeline on the real CVM-H slices, asserts `(N_z = 29)`, `(z[0] = -70 000)`, `(z[-1] = 0)`, and that `assert (Vp_min ≥ 800) and (Vp_max ≤ 8000)`.

Edge Cases to Handle

- **Filename vs header depth mismatch** — header is the truth (see §1).
- **Velocity = 0 cells** — CVM-H uses `0.0` to mark masked cells in some rasters; the schema reserves NaN for missing. Convert `0.0` → NaN only when **all four columns** are zero (not just Vp); flag in stats.
- **Lon order reversed in some slices** — defensively sort by lon ascending after parsing each slice.

Acceptance Criteria

- ☐ `python -m data_projection.build_velocity_cvmh --raw-dir data_velocity --out-path data_projected/velocity_safs.h5` succeeds.
- ☐ `h5dump -A` on the output shows `schema_version = "data_projection_v1"`, `crs = "EPSG:32611"`, `z_positive = "elevation"`, and `oob_policy` set.
- ☐ Output `(x, y, z)` axes are strictly monotone increasing.

☐ `pytest -q test_data_projection.py` is green (synthetic fixtures + the smoke test).
☐ No edit to existing seas / preprocess files outside `code_preprocess/data_projection/`.

Dependencies

- Depends on: Phase 1.
- Required by: Phase 4 (runtime loader needs a real sidecar to test).

Phase 3 — Runtime Loader & Interpolant (`DataField3D`)

Goal

A header-only C++ class that loads a schema-v1 sidecar and evaluates the trilinear interpolant of any of its `/fields/<name>` at arbitrary `(x, y, z)` in UTM 11 N. Stateless after construction; thread-safe for concurrent reads.

Files to Create

- `miniapps/seas/io/data_field_3d.hpp` — declaration + inline impl (header-only because of templates over `real_t`).
- `miniapps/seas/io/data_field_3d.cpp` — only if the HDF5 implementation cost forces moving non-template code out of the header.
- `miniapps/seas/test/test_data_field_3d.cpp` — unit tests (gtest pattern matching existing `test/test_*.cpp`).

Files to Modify

- `miniapps/seas/CMakeLists.txt` — add the new sources and link `${HDF5_C_LIBRARIES}` (already a transitive dep via MFEM but cite explicitly so a configure-time `find_package(HDF5 REQUIRED)` is enforced).
- `miniapps/seas/Makefile` — same.

Detailed Requirements

1. **Class definition.** Approximate signature:

```cpp
namespace mfem::seas {
enum class OOBPolicy : int { Clamp, Abort, ConstantFill };

class DataField3D
{
public:
    /// Load a single named field from a schema-v1 HDF5 sidecar.
    /// Verifies schema_version, crs, z_positive at load time and
    /// MFEM_ABORT()s with a precise error on mismatch.
    DataField3D(const std::string& sidecar_path,
                const std::string& field_name,
                OOBPolicy oob = OOBPolicy::Clamp,
                real_t fill_value = 0.0);

    /// Evaluate the trilinear interpolant at (x, y, z) in canonical
    /// CRS (UTM 11 N, m). Out-of-bbox dispatches per OOBPolicy.
    /// Returns std::numeric_limits<real_t>::quiet_NaN() if any
    /// stencil corner is NaN AND policy != Clamp.
```

```
    real_t Evaluate(real_t x, real_t y, real_t z) const;

    /// Bbox in canonical CRS for caller-side culling.
    const std::array<real_t, 6>& BBox() const { return bbox_; }

    /// Field metadata.
    const std::string& FieldName() const { return field_name_; }
    const std::string& Units()     const { return units_; }
    int NumOOBHits() const { return oob_hits_; }
    int NumNanHits() const { return nan_hits_; }

private:
    std::vector<real_t> x_, y_, z_;          // monotone axes
    mfem::Array3D<real_t> data_;              // (Nx, Ny, Nz)
    std::array<real_t, 6> bbox_;
    OOBPolicy oob_policy_;
    real_t fill_value_;
    std::string field_name_, units_;
    mutable std::atomic<int> oob_hits_{0}, nan_hits_{0};

    int find_index_(const std::vector<real_t>& axis, real_t v) const;
};
} // namespace mfem::seas
```

2. **Schema check.** At load time read root attrs and verify:

```
MFEM_VERIFY(schema_version == "data_projection_v1",
            "DataField3D: unexpected schema_version='" <<
            schema_version << "', expected 'data_projection_v1'");
MFEM_VERIFY(crs == "EPSG:32611",
            "DataField3D: only EPSG:32611 supported, got '" <<
            crs << "'");
MFEM_VERIFY(z_positive == "elevation",
            "DataField3D: only z_positive='elevation' supported");
```

3. **Trilinear evaluation.** Standard formula:
   - `i = bsearch(x_, x)`, `j = bsearch(y_, y)`, `k = bsearch(z_, z)`.
   - Local coords `u = (x - x_[i]) / (x_[i+1] - x_[i])`, etc.
   - `f = (1-u)(1-v)(1-w)f_000 + u(1-v)(1-w)f_100 + … + uvw f_111`.
   - If any `f_ijk` is NaN: `++nan_hits_`; under `Clamp` substitute the in-bbox edge value; under `Abort` `MFEM_ABORT`; under `ConstantFill` return `fill_value_`.
   - If `(x, y, z)` is outside `[x_[0], x_[Nx-1]]^3`: `++oob_hits_`; under `Clamp` snap to the nearest axis bin; under `Abort` `MFEM_ABORT` with the offending coord; under `ConstantFill` return `fill_value_`.
4. **Binary search.** `find_index_` uses `std::upper_bound` on `axis.begin()..end()-1` and clamps to `[0, axis.size()-2]`. The `-1` on `end()` ensures `i+1` is always valid.
5. **HDF5 dependency.** Use the C API (`H5Fopen`, `H5Dread`, `H5Aread`) not `HighFive` to avoid a new dep. MFEM already vendors HDF5 use via its ParaView writer; copy the include pattern from `mfem/general/binaryio.hpp`.
6. **Thread safety.** All reads are const. `oob_hits_` and `nan_hits_` are `std::atomic<int>` so the parallel `ParGridFunction::ProjectCoefficient` loop can update them safely.

Edge Cases to Handle

- **Sidecar missing**: `MFEM_ABORT("DataField3D: cannot open sidecar '<path>'")`.
- **Field name not present**: list available `/fields/*` in the abort message.
- **Single-cell axis** (`Nx == 1`): treat as exact match required (any off-axis query is OOB).
- **NaN at every stencil corner with `Clamp`**: walk outward until a non-NaN sample is found, up to a configurable radius `kClampSearchRadius = 4` cells. If no non-NaN sample is found, fall back to `fill_value_`.
- **Partial NaN stencil** (1–7 of 8 corners NaN): under `Clamp`, replace each NaN corner with the nearest non-NaN corner; the bilinear/trilinear weights are unchanged.

Interfaces

- `DataField3D::Evaluate(x,y,z)` is the only runtime entry point.
- Re-exported through `miniapps/seas/io/data_field_3d.hpp` so any consumer (Phase 4, 5) only needs `#include "io/data_field_3d.hpp"`.

Acceptance Criteria

- ☐ `test_data_field_3d` covers: load, schema-mismatch abort, exact-axis query, midpoint query (analytic check), corner query, OOB clamp, OOB abort, NaN clamp, NaN abort.
- ☐ `make test-data-field-3d` is green at np = 1.
- ☐ No edits to `bp5/`, `bp1/`, `bp2/`, `domain/`, `friction/`, `dynamic/`. (Phase 4 will do those edits, additively.)

Dependencies

- Depends on: Phase 1, Phase 2 (for at least one real sidecar to test against).
- Required by: Phase 4, 5.

Phase 4 — Mesh Projector (`FieldCoefficient` + `FieldProjector`)

Goal

Bind a `DataField3D` to MFEM via the standard `Coefficient` interface so that `ParGridFunction::ProjectCoefficient` does the bulk-DOF projection in one call. Add a fault-DOF specialisation that walks the fault quadrature points (already enumerated by `FaultBasis` / `FaultGeometry`) and evaluates `DataField3D` at each.

Files to Create

- `miniapps/seas/io/field_coefficient.hpp` — `FieldCoefficient`, `FieldVectorCoefficient`, `FieldProjector` (header-only).

Files to Modify

- `miniapps/seas/CMakeLists.txt` / `Makefile` — register the new header.

Detailed Requirements

1. **`FieldCoefficient` (scalar).** Approximate signature:

```
class FieldCoefficient : public mfem::Coefficient
{
public:
    FieldCoefficient(const DataField3D& field, real_t scale = 1.0,
                     real_t floor = -std::numeric_limits<real_t>::infinity())
```

```
        : field_(field), scale_(scale), floor_(floor) {}

    real_t Eval(mfem::ElementTransformation& T,
                const mfem::IntegrationPoint& ip) override
    {
        Vector x;
        T.Transform(ip, x);
        real_t v = scale_ * field_.Evaluate(x[0], x[1], x[2]);
        return std::max(v, floor_);
    }

private:
    const DataField3D& field_;
    real_t scale_, floor_;
};
```

`floor_` implements the `V_s_floor` requirement from Constraints §3 without modifying the sidecar. **Convention:** consumers compose floors and unit conversions at the `FieldCoefficient` boundary (e.g. to project $\mu = \rho$ `V_s`^{2}, build a `FieldCoefficient(Vs, 1, V_s_floor)`, a `FieldCoefficient(rho)`, then derive μ via `mfem::ProductCoefficient`

- `mfem::ProductCoefficient`).

2. **`FieldProjector` (driver-side helper).** Signature:

```
struct FieldProjectionResult
{
    std::shared_ptr<mfem::ParGridFunction> gf;
    int n_oob, n_nan, n_floored;
    real_t min_value, max_value;
};

class FieldProjector
{
public:
    /// Project a single named field onto the H1(p) finite element
    /// space `target_fes`.
    static FieldProjectionResult Project(
        const DataField3D&          field,
        mfem::ParFiniteElementSpace& target_fes,
        real_t scale = 1.0,
        real_t floor = -1e300);

    /// Convenience for the velocity field set: returns three GFs in
    /// the order {ρ, λ, μ} given Vp, Vs, ρ in the sidecar. λ and μ
    /// are derived: μ = ρ V_s², λ = ρ V_p² − 2 μ.
    struct VelocityFields {
        std::shared_ptr<mfem::ParGridFunction> rho, lambda, mu;
        int n_floored_vs, n_floored_vp;
    };
    static VelocityFields ProjectVelocity(
        const std::string&           sidecar_path,
        mfem::ParFiniteElementSpace& target_fes,
        real_t vs_floor_mps  = 1500.0,
        real_t vp_floor_mps  = 2000.0,
        OOBPolicy oob        = OOBPolicy::Clamp);
```

```
};
```

3. **ParaView snapshot.** After `Project` / `ProjectVelocity`, the driver MUST write a ParaView VTU pair for visual QA before the simulation starts:

   `<out_dir>/projected_fields/<sidecar-stem>_<mesh-tag>.{pvtu,vtu}`

   This is invoked from the driver, not from `FieldProjector`, to keep the projector pure. The driver path is added in Phase 5.

4. **OOB / NaN reporting.** `Project` rank-reduces `n_oob` and `n_nan` via `MPI_Reduce(MPI_SUM, root=0)` and prints a one-line summary on rank 0:

   `[FieldProjector] field='Vs' oob=0 nan=0 floored=124  min=1500.00 max=4612.31  m/s`

   Aborts loudly if `n_oob > 0` and the field's load policy is `OOBPolicy::Abort` (the abort already happens inside `DataField3D::Evaluate`, but the post-hoc reduce is informational).

Edge Cases to Handle

- **Floor activation count.** Track `n_floored = #{q : raw value < floor}` per field per rank.
- **Order of FE space.** `Project` must work for `H1(p)` for any `p ∈ {1, 2, 3}` and for L^2(p) (DG). Use `ParGridFunction::ProjectCoefficient` which does the right thing in both cases.
- **Mesh element-type heterogeneity.** Future-proof: do not assume tetrahedra. The CVM-H projection is currently only used on tet meshes, but the projector should work on hex / wedge / prism via the `IntegrationPoint::Transform` abstraction.

Acceptance Criteria

- ☐ On the 2000 m mesh with the real CVM-H sidecar, `ProjectVelocity` returns three GFs with `min_value > 0` after flooring.
- ☐ ParaView snapshot opens and shows the velocity model textured onto the mesh, with a clean clamp boundary outside the CVM-H hull.
- ☐ `mpirun -np 4 test_field_projector_smoke` runs and reports identical (ρ, λ, μ) min/max as np=1 (rank-reduce works).

Dependencies

- Depends on: Phase 3.
- Required by: Phase 5.

Phase 5 — Consumer Adapters (`Wave0perator`, fault DOFs, friction)

Goal

Wire the projected (`ρ, λ, μ`) and (later) (`σ0, p`) `ParGridFunctions` into the dynamic-rupture stack. After this phase a TPV102 driver invoked with `--velocity-sidecar <path.h5>` initialises with a heterogeneous medium and the simulation runs to completion at np $\geq$ 1 without a single edit to BP5 / TPV104 / TPV205 source.

Files to Create

- `miniapps/seas/dynamic/heterogeneous_material.hpp` — `MaterialField` struct holding the three projected `ParGridFunctions` plus per-DOF cached (`Z_p, Z_s, η_p, η_s`).
- `miniapps/seas/dynamic/heterogeneous_material.cpp` — derived-quantity computation.

Files to Modify

- `miniapps/seas/dynamic/wave_operator.hpp` — add a second constructor:

```
WaveOperator(MeshType& mesh, int order,
             const MaterialField& material,
             const BoundaryConfig& bc);
```

  The original `(λ, μ, ρ)` constructor remains and forwards to a no-op `MaterialField` (constant field) so existing call sites are byte-identical.

- `miniapps/seas/dynamic/wave_operator.inl` — replace the scalar `lambda_, mu_, rho_` reads inside `Mult` and the ADER recursion with a `material_.At(elem, dof)` accessor that dispatches to (a) the scalar values when material is constant or (b) the GridFunction-backed per-DOF values otherwise. **Critical:** must produce byte-identical output to the existing path when the material is constant.

- `miniapps/seas/dynamic/fault_face_flux.cpp` — add an `InitializeImpedancesFromMaterial(DOFData* dof, const MaterialField&, const FaultGeometry&)` helper that fills `Zp_plus, Zp_minus, …` using the per-DOF $\lambda/\mu/\rho$ values at each fault QP.

- `miniapps/seas/drivers/tpv102_driver.cpp` (and only this driver in the first cut — TPV104 / TPV205 / BP5 stay unchanged) — add CLI flag `--velocity-sidecar PATH` that, when set, builds a `MaterialField` via `FieldProjector::ProjectVelocity` and passes it to the new `WaveOperator` ctor. Default off → byte-identical to today.

Detailed Requirements

1. **`MaterialField` struct.** Approximate definition:

```
struct MaterialField
{
    enum class Mode : int { Constant, GridFunction };
    Mode mode = Mode::Constant;

    // Scalar fallback (mode == Constant)
    real_t lambda_const = 0, mu_const = 0, rho_const = 0;

    // Heterogeneous (mode == GridFunction). Same FES across all three.
    std::shared_ptr<mfem::ParGridFunction> rho_gf, lambda_gf, mu_gf;

    /// Element-and-DOF-local accessor used inside ADER assembly.
    inline void At(int elem, int dof,
                   real_t& lambda_out, real_t& mu_out, real_t& rho_out) const;

    /// Per-element minimum c_p (for CFL): max over DOFs in elem.
    real_t MaxCpInElement(int elem) const;
};
```

   `At` must be `inline` and branch-free (a function pointer or a compile-time tag) on the hot path.

2. **`WaveOperator` ctor changes.** New ctor:

```
WaveOperator(MeshType& mesh, int order,
             const MaterialField& material,
             const BoundaryConfig& bc);
```

   stores `material_` by value. Old ctor:

```
WaveOperator(MeshType& mesh, int order,
             real_t lambda, real_t mu, real_t rho,
```

```
            const BoundaryConfig& bc)
    : WaveOperator(mesh, order,
                   MaterialField{Mode::Constant, lambda, mu, rho, ...},
                   bc) {}
```

so the same downstream code path runs in both cases.

3. **ADER assembly under heterogeneous material.** In `wave_operator.inl::Mult` and the ADER Cauchy–Kovalevskaya recursion (see `dynamic_rupture_plan_v4.md` §4) every read of (λ, μ, ρ) must be replaced with `material_.At(elem, dof, λ, μ, ρ)`. The flux matrices (`A_x, A_y, A_z`) are **DOF-local**: each quadrature point sees its own `(c_p, c_s)`.

   Conservation check: when the field is constant the old per-element precomputed A_x etc. are still recomputable from (λ, μ, ρ) at any DOF. So the existing precomputation can stay; we only branch to the heterogeneous DOF-local path when `material_.mode == GridFunction`.

4. **CFL.** Replace the constant `c_p` in `WaveOperator::ComputeMaxDt` with `material_.MaxCpInElement(elem)` element-wise; reduce via `MPI_Allreduce(MIN, dt_local)`.

5. **Fault impedances.** `InitializeImpedancesFromMaterial` walks the fault QPs (handle from `FaultBasis::GetFaultQPs()`), evaluates the three GFs at each QP (which lives on a face: average of the +/- sides, see `de la Puente 2009 §3.3`), and writes

```
dof.Zp_plus  = ρ⁺ · √((λ⁺+2μ⁺)/ρ⁺) ;
dof.Zp_minus = ρ⁻ · √((λ⁻+2μ⁻)/ρ⁻);
dof.Zs_plus  = ρ⁺ · √(μ⁺/ρ⁺);
dof.Zs_minus = ρ⁻ · √(μ⁻/ρ⁻);
dof.eta_p    = (Zp_plus*Zp_minus)/(Zp_plus+Zp_minus);
dof.eta_s    = (Zs_plus*Zs_minus)/(Zs_plus+Zs_minus);
```

   This is purely additive — TPV102's existing constant initialization becomes a special case where $\rho^+ = \rho^-$ etc.

6. **Driver wiring (TPV102 only in this phase).**
   - Parse `--velocity-sidecar PATH`.
   - If unset: existing behaviour (byte identical).
   - If set:
     1. Build `DataField3D vp(path, "Vp", Clamp); vs(path, "Vs", Clamp); rho(path, "density", Clamp);`.
     2. `MaterialField mat = FieldProjector::ProjectVelocity(path, target_fes, vs_floor=1500, vp_floor=2000);`.
     3. Pass `mat` to the new `WaveOperator` ctor.
     4. `InitializeImpedancesFromMaterial(dof_data, mat, fault_geom);`.
     5. After the `WaveOperator` is constructed, write the ParaView snapshot.

7. **No-op for stress / pore pressure in this phase.** The schema supports them; the consumer adapters for `InitializeStateTotal` and the friction law are deliberately deferred to the next plan revision so the velocity work can ship in isolation.

Edge Cases to Handle

- **Mesh DOFs outside the CVM-H hull (most of the doubled-domain bulk).** Clamp policy returns the edge values (still physical: a representative basement velocity). Confirm by inspecting the ParaView snapshot.
- **Vs floor activation** must not break TPV102 spec compliance when the projection is OFF. Verified by the byte-identical check at `--velocity-sidecar` unset.

- **Fault QP straddling the hull edge.** The fault span (≈ 258 km along the SAF strike) is much larger than the CVM-H hull (≈ 250 km diagonal); some QPs at the NW / SE tips will land outside. Document the fraction in the projection summary; abort if > 5 %.

Acceptance Criteria

- ☐ `mpirun -np 1 tpv102_driver --mesh tpv102_mesh.msh` (no `--velocity-sidecar`) produces output that matches a stored reference file byte-for-byte (regression guard).
- ☐ `mpirun -np 4 tpv102_driver --mesh safs_fault_box_nwcut_2000m.msh --velocity-sidecar data_projected/velocity_safs.h5` runs to completion without abort, and the rupture front in the ParaView output is visibly slowed in the high-Vp basement layer.
- ☐ CFL reduces correctly: Δt_CFL reported by the driver shrinks compared to the homogeneous TPV102 baseline (since the CVM-H surface layer has lower V_s).
- ☐ Existing test suite (`make test`) is green.

Dependencies

- Depends on: Phase 4.
- Required by: any future per-DOF-stress / pore-pressure consumer.

Phase 6 — Generalisation Hooks (deferred, not implemented in v1)

This is a **roadmap**, not an action item. After Phase 5 ships, the following extensions land as their own plan v2:

- **Stress field consumer.** Add an `InitializeStateTotal_Field` overload in `dynamic/tpv102_setup_total.hpp` that takes six `ParGridFunctions` (σ_xx, σ_yy, σ_zz, σ_xy, σ_yz, σ_xz) and writes them into bulk Q at every DOF. The existing constant- pre-stress overload stays.
- **Pore pressure consumer.** Add a `friction/dieterich_ruina.hpp` hook **without** editing that file (CLAUDE.md C2): pass an optional `pore_pressure_gf` through `DOFData` and have the friction call read effective normal stress = σ_n − p only when `pore_pressure_gf` is non-null. Wire the consumer in a new `dynamic/effective_stress.hpp`.
- **Spatially varying friction parameters.** Same pattern: project `a(x)`, `Dc(x)` GFs onto the fault and have `InitializeFaultDOFs_*` read them when present.
- **Generic config plumbing.** Extend `config/seas_config.hpp` / `seas_config_parser.hpp` so a TOML stanza

  ```toml
  [data_projection]
  velocity = "data_projected/velocity_safs.h5"
  stress   = "data_projected/stress_cfm_safs.h5"
  pore     = "data_projected/pore_baseline.h5"
  ```

  drives the projection automatically without per-driver CLI churn.
- **Higher-order interpolation.** A `DataField3D::EvaluateCubic` variant via tricubic / monotone Hermite for fields where C^1 continuity matters (stress).

Testing Strategy

Phase	Test type	What it covers
1	doc lint	schema doc covers every attribute used in Phases 2 / 3

Phase	Test type	What it covers
2	pytest unit + smoke	reader correctness, CRS round-trip, sidecar round-trip
3	gtest unit	trilinear evaluation, OOB / NaN policies, schema-mismatch
4	gtest unit + np=1/4	`ProjectCoefficient` round-trip, MPI reduce of OOB / NaN
5	TPV102 reg + smoke	byte-identical when off; runs to completion when on

Validation against analytics

For Phase 3 (interpolant) the reference solution is closed-form trilinear: place a fixture field `f(x,y,z) = ax + by + cz + d` on a 3 × 3 × 3 grid; assert `Evaluate(x,y,z)` equals `f(x,y,z)` to 1e-12 at 1024 random in-bbox points. Linear functions are exact under trilinear interpolation; this is the standard convergence-zero check.

For Phase 5 the validation pathway is **CFL drop** (well-defined, checkable in O(1) sim time) plus a **2-D slice comparison** of projected ρ at z = −5 km against the raw CVM-H slice — root-mean-square deviation across the in-hull region must be < 0.5 % (numerical residue of the rectilinear-resampling step in Phase 2).

Risk Assessment

High risk

- **Coordinate-frame slip.** The .ts files are UTM 11 N, the meshes inherit that, the velocity raw is geographic. A single cut-and-paste error in `geographic_to_utm11n` (e.g. `Transformer.from_crs(..., always_xy=False)`) produces a silently shifted result of order 100 km. Phase 2 must include the round-trip test `test_geographic_to_utm11n_round_trip` and a corner-anchor test that specifically verifies `(Lon1, Lat1)` → known UTM corner ±0.1 m.
- **`WaveOperator` heterogeneity refactor breaking TPV104 / TPV205 / BP5.** The "constant material is a special case of MaterialField" approach minimises risk, but every existing test that hashes `Mult(Q)` output is a tripwire. The Phase 5 acceptance criterion "byte-identical when off" exists for exactly this reason and must be enforced before merge.

Medium risk

- **Vs floor poisons material physics.** A 1500 m/s floor is fine for rupture dynamics but spuriously stiffens water-saturated layers. Document explicitly that the floor is a CFL band-aid and surface the count of floored DOFs in every run.
- **HDF5 portability.** Building on Frontera with the project's toolchain (ICX 2023.1 + IMPI 2021.9 + GCC 9.1, see `reference_seissol_frontera.md`) requires that HDF5 is linked consistently with MFEM. Check `mfem-config --libs` for `-lhdf5` before merging.

Low risk

- **CVM-H mask ambiguity.** Phase 2 §1 documents the `0.0` → NaN conversion rule explicitly; if CVM-H later changes its mask convention the rule is in one place.

Summary of New Files

```
miniapps/seas/document/features_dev/
  data_projection_feature_plan_v1.md       (THIS FILE)
  data_projection_schema_v1.md             (Phase 1)
```

```
safs/project_7.0_alternative/code_preprocess/data_projection/
  __init__.py                              (Phase 2)
  raw_readers.py
  crs.py
  sidecar.py
  build_velocity_cvmh.py
  test_data_projection.py

safs/project_7.0_alternative/data_projected/
  velocity_safs.h5                         (Phase 2 output, generated)

miniapps/seas/io/
  data_field_3d.hpp                        (Phase 3)
  data_field_3d.cpp                        (Phase 3, optional)
  field_coefficient.hpp                    (Phase 4)

miniapps/seas/dynamic/
  heterogeneous_material.hpp               (Phase 5)
  heterogeneous_material.cpp               (Phase 5)

miniapps/seas/test/
  test_data_field_3d.cpp                   (Phase 3)
  test_field_projector.cpp                 (Phase 4)
  test_tpv102_with_velocity_sidecar.cpp    (Phase 5 smoke)
```

Files Modified (additive only)

```
miniapps/seas/CMakeLists.txt                          (Phase 3–5: add sources)
miniapps/seas/Makefile                                (Phase 3–5: add sources)
miniapps/seas/dynamic/wave_operator.hpp               (Phase 5: new ctor)
miniapps/seas/dynamic/wave_operator.inl               (Phase 5: At()-dispatch)
miniapps/seas/dynamic/fault_face_flux.cpp             (Phase 5: impedance helper)
miniapps/seas/drivers/tpv102_driver.cpp               (Phase 5: CLI flag)
```

No edits to: `bp5/`, `bp1/`, `bp2/`, `domain/`, `friction/dieterich_ruina.hpp`, `safs/CFM_data_step/`, any `.toml` example, or any other driver. C2 invariant preserved.